

Department of Electrical and Information Engineering

University of Ruhuna

EC8208 – Software Architecture

Continuous Assessment 01

A Smart Public Transport Tracking and E-Ticketing System for Sri Lanka

Group Members

EG/2020/4029 – Kulathilaka W.A.S.P

EG/2020/4128 – Rajakaruna R.H.M.K.M

EG/2020/4148 – Randima H.G.L

EG/2020/4202 – Senarathna G.M.L.D

EG/2020/4351 – Perera M.T.T.M

Submission date

13/07/2026

Department of Electrical and Information Engineering

University of Ruhuna

Contents

1 Introduction **2**

1.1 Background of the Selected Problem 2

1.2 Problem Statement 2

1.3 Objectives of the Proposed System 3

2 Requirements Analysis **3**

2.1 Functional Requirements 3

2.2 Non-Functional Requirements 4

2.3 Stakeholder Identification 5

3 Architecture Selection **6**

3.1 Candidate Architecture Styles 6

3.2 Comparison of Candidate Styles 6

3.3 Selected Style and Justification 7

4 Architectural Design **7**

4.1 Context Diagram 7

4.2 Architecture Diagram 7

4.3 Component Diagram 10

4.4 Deployment Diagram 11

4.5 Class Diagram 11

4.6 Data Flow Diagram (Level 1) 13

5 Technology Stack Selection **14**

6 Quality Attributes **15**

6.1 Scalability 15

6.2 Performance 15

6.3 Security 15

6.4 Availability 16

6.5 Maintainability 16

6.6 Reliability 16

7 Conclusion **16**

1. Introduction

1.1 Background of the Selected Problem

Public transport is the backbone of daily mobility in Sri Lanka. More than half of all motorised passenger trips in the country are carried by buses operated by the Sri Lanka Transport Board (SLTB) and thousands of private operators, supplemented by the commuter rail network of Sri Lanka Railways. Despite this heavy dependence, the passenger experience remains largely analogue: commuters wait at halts with no reliable information about when the next bus will arrive, tickets are issued as small paper slips against cash payments, seat reservation for long-distance services is fragmented across ad-hoc phone calls and third-party counters, and operators have almost no digital visibility into fleet utilisation or revenue leakage.

The consequences are felt by every stakeholder. Passengers lose productive time waiting for vehicles whose arrival cannot be predicted, especially during peak congestion in the Colombo Metropolitan Region. Cash-only fare collection creates disputes over change, enables fare evasion and under-reporting, and became a hygiene concern in recent years. Regulators such as the National Transport Commission (NTC) lack accurate operational data on which to base route licensing and subsidy decisions. Meanwhile, smartphone penetration and 4G coverage in Sri Lanka have reached levels that make a digital, real-time solution both technically feasible and socially inclusive, particularly when complemented with QR-based payments such as LankaQR.

This report proposes TransitLK, a cloud-native software platform that provides real-time vehicle tracking, journey planning, cashless e-ticketing with QR validation, and operational analytics for public transport in Sri Lanka. The remainder of this report analyses the requirements of such a system, evaluates candidate architecture styles, presents a detailed architectural design based on the microservices style, selects a suitable technology stack, and discusses how the design satisfies the key quality attributes.

1.2 Problem Statement

Commuters, operators, and regulators of Sri Lankan public transport currently lack a unified digital platform for real-time vehicle tracking and cashless ticketing. The absence of live location information leads to unpredictable waiting times and poor journey planning; manual, cash-based ticketing causes revenue leakage, fare disputes, and long boarding times; and the lack of consolidated operational data prevents evidence-based decisions on scheduling, route licensing, and fleet management. A scalable, secure, and highly available software system is required to close these gaps while operating reliably under the bursty loads and intermittent connectivity conditions typical of a nationwide transport network.

1.3 Objectives of the Proposed System

- To provide passengers with real-time vehicle locations and accurate estimated times of arrival (ETA) for buses and trains on a live map.
- To enable end-to-end cashless ticketing: fare calculation, seat booking for long-distance services, secure online payment, and QR-code e-ticket issuance and validation.
- To give fleet operators dashboards for fleet monitoring, schedule management, occupancy analysis, and revenue reporting.
- To supply the transport regulator with reliable, aggregated operational analytics that support route licensing and policy decisions.
- To deliver these capabilities through a scalable, secure, and maintainable cloud-native architecture capable of handling peak-hour demand spikes and continuous GPS telemetry from thousands of vehicles.

2. Requirements Analysis

2.1 Functional Requirements

The functional requirements are grouped by the major capabilities of the system.

FR1 – User Registration and Authentication:
Passengers, conductors, operators, and administrators shall be able to register and log in securely using email/phone number and password, with OTP verification for account activation and role-based access to functionality.

FR2 – Route and Schedule Search:
Passengers shall be able to search routes by origin, destination, and time; the system shall return matching trips with fares, departure times, and current seat availability.

FR3 – Real-Time Vehicle Tracking:
The system shall ingest GPS telemetry from onboard devices every 5–10 seconds and display live vehicle positions and computed ETAs to passengers on a map.

FR4 – Ticket Booking and Fare Calculation:
Passengers shall be able to purchase point-to-point tickets and reserve seats on long-distance services; fares shall be calculated automatically from the fare-stage model of the selected route.

FR5 – Online Payment Processing:
The system shall accept payments via debit/credit cards, LankaQR, and mobile wallets

through certified payment gateways, and shall support an in-app stored-value wallet with top-up and refunds.

FR6 – QR E-Ticket Issuance and Validation:
On successful payment, the system shall issue a digitally signed QR e-ticket; conductors shall validate tickets offline-tolerantly using the validator app, and each ticket shall be usable exactly once.

FR7 – Notifications:
The system shall notify passengers of booking confirmations, payment receipts, service delays, and vehicle-approaching alerts via push notifications and SMS.

FR8 – Fleet and Schedule Management:
Operators shall be able to register vehicles, assign them to routes and trips, publish timetables, and mark service disruptions.

FR9 – Reporting and Analytics:
The system shall generate occupancy, revenue, punctuality, and route-performance reports for operators, and aggregated compliance analytics for the regulator.

FR10 – Trip Lifecycle Management:
Conductors/drivers shall be able to start and complete trips, and the system shall automatically reconcile issued tickets against completed trips.

2.2 Non-Functional Requirements

ID	Attribute	Requirement
NFR1	Performance	95% of API requests shall complete within 500 ms; live map updates shall reach clients within 3 s of a GPS ping; QR validation shall complete within 1 s.
NFR2	Scalability	The system shall scale horizontally to support 100,000+ concurrent users during peak hours and telemetry from 10,000+ vehicles without redesign.
NFR3	Availability	Core services (tracking, ticket validation) shall achieve 99.9% uptime; ticket validation shall degrade gracefully to offline signature verification when connectivity is lost.
NFR4	Security	All traffic shall be encrypted with TLS 1.3; passwords hashed with bcrypt; payment flows shall comply with PCI-DSS via tokenised gateways; QR tickets shall be digitally signed to prevent forgery.

ID	Attribute	Requirement
NFR5	Reliability	No confirmed payment shall result in a lost ticket; events shall be processed at-least-once with idempotent consumers; RPO $\leq$ 5 minutes, RTO $\leq$ 30 minutes.
NFR6	Maintainability	Services shall be independently deployable with automated CI/CD; new payment methods or vehicle types shall be addable without modifying unrelated services.
NFR7	Usability	The passenger app shall support Sinhala, Tamil, and English, and core journeys (search $\rightarrow$ pay $\rightarrow$ ticket) shall require no more than four screens.
NFR8	Interoperability	Public APIs shall follow REST/OpenAPI standards to allow future integration with third-party journey planners and government systems.

Table 1: Non-functional requirements of TransitLK

2.3 Stakeholder Identification

Stakeholder	Role / Interest in the System
Passengers	Primary end users; need reliable arrival information, easy booking, cashless payment, and trustworthy e-tickets.
Drivers / Conductors	Operate the onboard validator app; need fast, dependable QR validation and simple trip start/stop controls.
Fleet Operators (SLTB, private bus owners, Sri Lanka Railways)	Need fleet visibility, schedule management, occupancy and revenue analytics, and reduced fare leakage.
National Transport Commission (Regulator)	Needs accurate operational data for route licensing, subsidy allocation, and policy making.
Payment Providers (banks, LankaQR, mobile wallets)	Process transactions; require secure, standards-compliant integration.
System Administrators / DevOps Team	Operate, monitor, and maintain the platform; need observability and safe deployment mechanisms.
Development Team	Build and evolve the system; need a modular, well-documented architecture.

Table 2: Stakeholders of TransitLK

3. Architecture Selection

3.1 Candidate Architecture Styles

Four candidate styles were evaluated against the functional and non-functional requirements identified above.

Monolithic layered architecture. A single deployable application organised into presentation, business-logic, and data layers. It is simple to develop and test initially, but the whole application must be scaled and redeployed as one unit. A traffic spike in ticket sales would force scaling of the tracking logic as well, and a defect in one module can bring down the entire system — conflicting with NFR2 and NFR3.

Client–server (two/three tier). A thin client communicating with a central server and database. While suitable for a small booking office, it cannot absorb continuous telemetry from thousands of GPS devices or peak-hour booking surges, and offers no isolation between concerns.

Event-driven architecture (EDA). Components communicate through asynchronous events on a message broker. This style fits GPS telemetry and notification fan-out extremely well, but purely event-driven request/response interactions (e.g., searching a route) become awkward, and end-to-end reasoning is harder.

Microservices architecture. The system is decomposed into small, independently deployable services, each owning its data and communicating over lightweight protocols. It provides selective scaling (tracking can scale independently of ticketing), fault isolation, technology flexibility, and team autonomy, at the cost of higher operational complexity.

3.2 Comparison of Candidate Styles

Criterion	**Monolithic Layered**	**Client–Server**	**Event-Driven**	**Microservices**
Independent scalability	Poor	Poor	Good	Excellent
Fault isolation	Poor	Poor	Moderate	Excellent
Handles telemetry streams	Weak	Weak	Excellent	Good (with broker)
Deployment independence	None	None	Moderate	Excellent
Development simplicity	Excellent	Excellent	Moderate	Moderate–Low

Criterion	Monolithic Layered	Client–Server	Event-Driven	Microservices
Operational complexity	Low	Low	Moderate	High
Fit to NFR1–NFR6	Low	Low	Moderate	High

Table 3: Comparison of candidate architecture styles

3.3 Selected Style and Justification

TransitLK adopts a microservices architecture complemented by an event-driven backbone. Synchronous, user-facing interactions (searching routes, booking tickets, validating QR codes) are handled by REST/gRPC calls routed through an API gateway, while high-volume, asynchronous flows (GPS telemetry, ticket events, payment events, notifications) travel over an Apache Kafka event bus. This hybrid resolves the weaknesses of each pure style: the microservices decomposition delivers the independent scalability, fault isolation, and maintainability demanded by NFR2, NFR3, and NFR6, while the event backbone absorbs telemetry bursts and decouples producers from consumers, satisfying NFR1 and NFR5.

The decomposition follows domain-driven design: each service corresponds to a bounded context (identity, routes and schedules, tracking, ticketing, payments, notifications, analytics) and owns its database (database-per-service pattern), preventing hidden coupling through shared schemas. The added operational complexity of microservices is mitigated by Kubernetes orchestration, centralised observability, and managed cloud services, as described in Sections 4 and 5.

4. Architectural Design

4.1 Context Diagram

The context diagram positions TransitLK as a single system and shows its interactions with external actors: passengers using the mobile/web apps, conductors using the validator app, fleet operators and the regulator consuming dashboards, GPS/IoT devices streaming telemetry, payment gateways, and messaging providers.

4.2 Architecture Diagram

The overall architecture comprises four tiers. The client tier contains the passenger app, web portal, conductor validator app, and operator/regulator dashboards. The edge tier terminates TLS at a CDN/load balancer and routes requests through an API gateway that performs authenti-

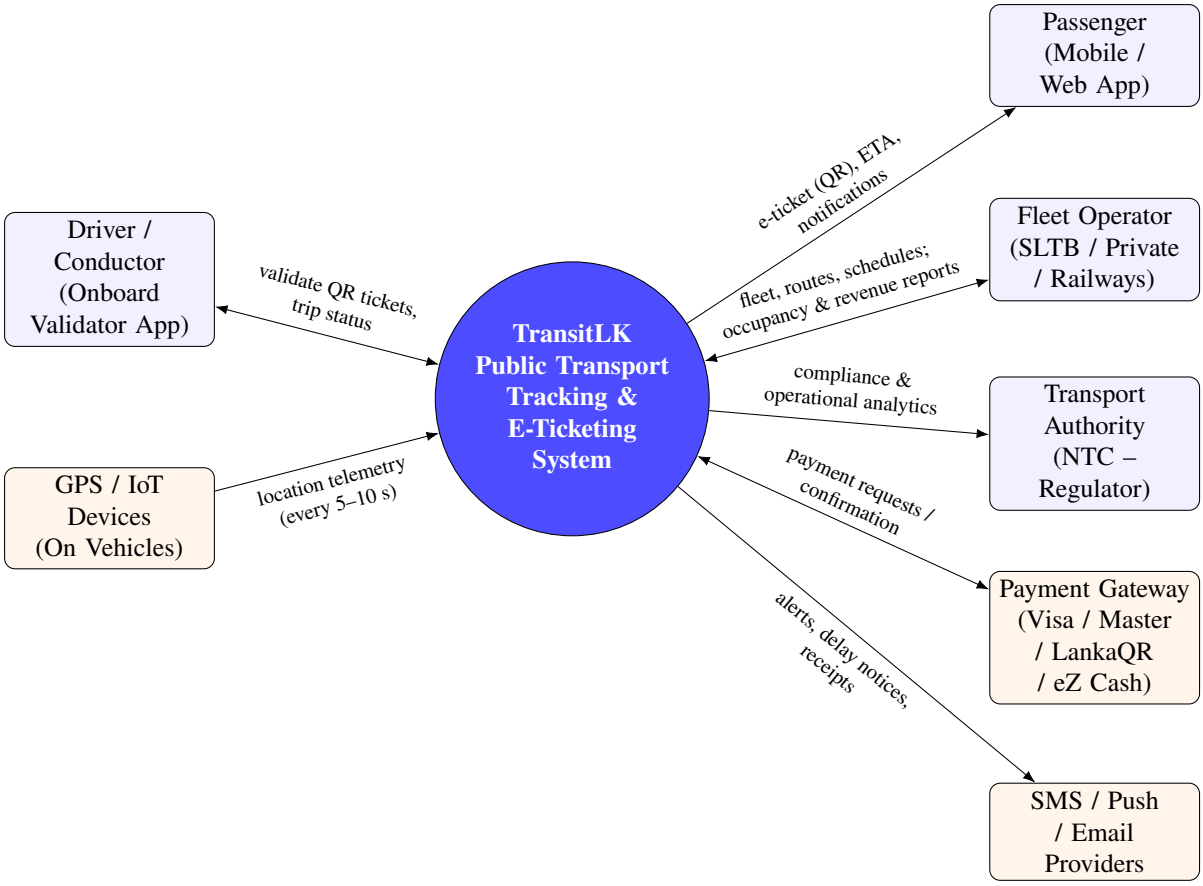


Figure 1: Context diagram of the TransitLK system

cation, rate limiting, and routing. The microservices tier hosts seven independently deployable services on Kubernetes. A Kafka event backbone decouples telemetry and business events, and the data tier applies polyglot persistence: PostgreSQL for transactional data, Redis for live vehicle positions and sessions, TimescaleDB for historical telemetry, and object storage for generated reports.

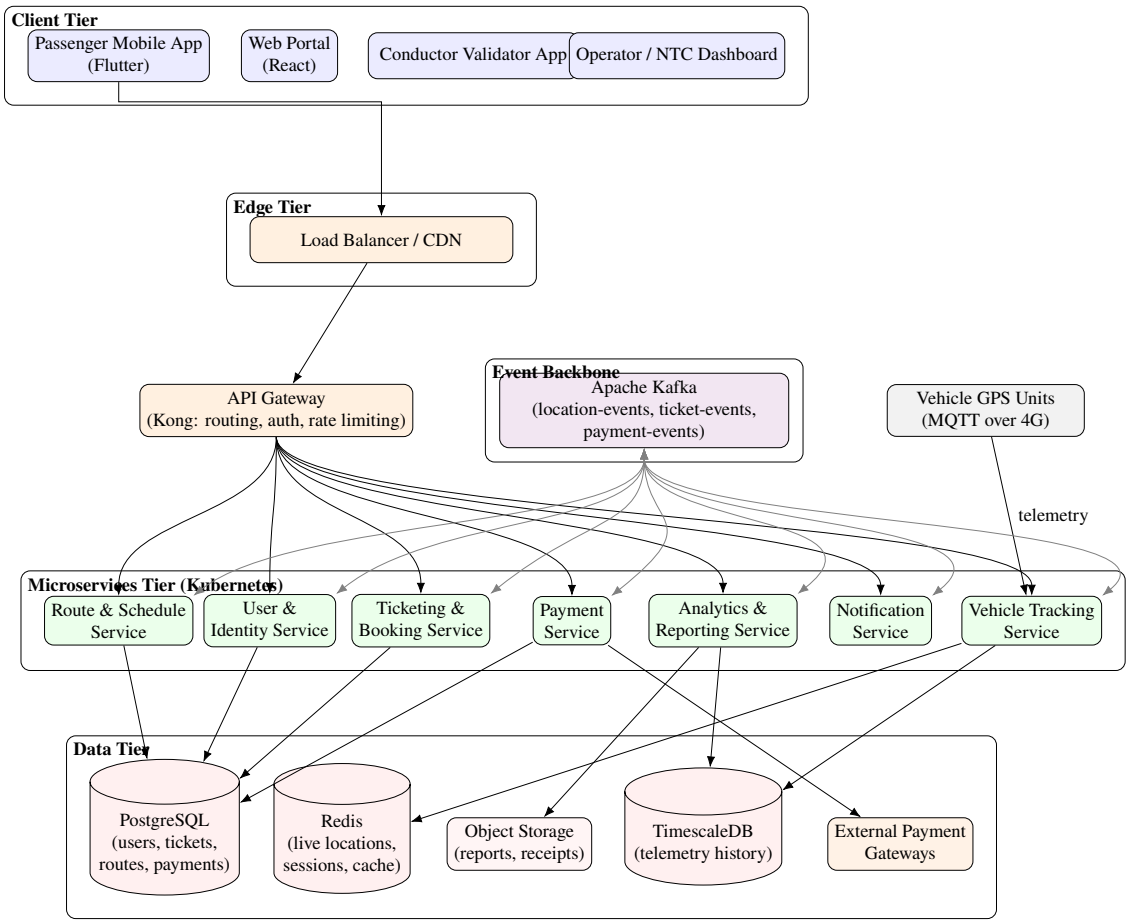


Figure 2: High-level microservices architecture of TransitLK

4.3 Component Diagram

The component diagram details the internal components of the core services and their interfaces. Synchronous REST/WebSocket interfaces are exposed through the API gateway; inter-service calls that require low latency (booking → fare/seat information, booking → payment) use gRPC; and all state-changing services publish events to Kafka topics consumed by the notification and analytics services.

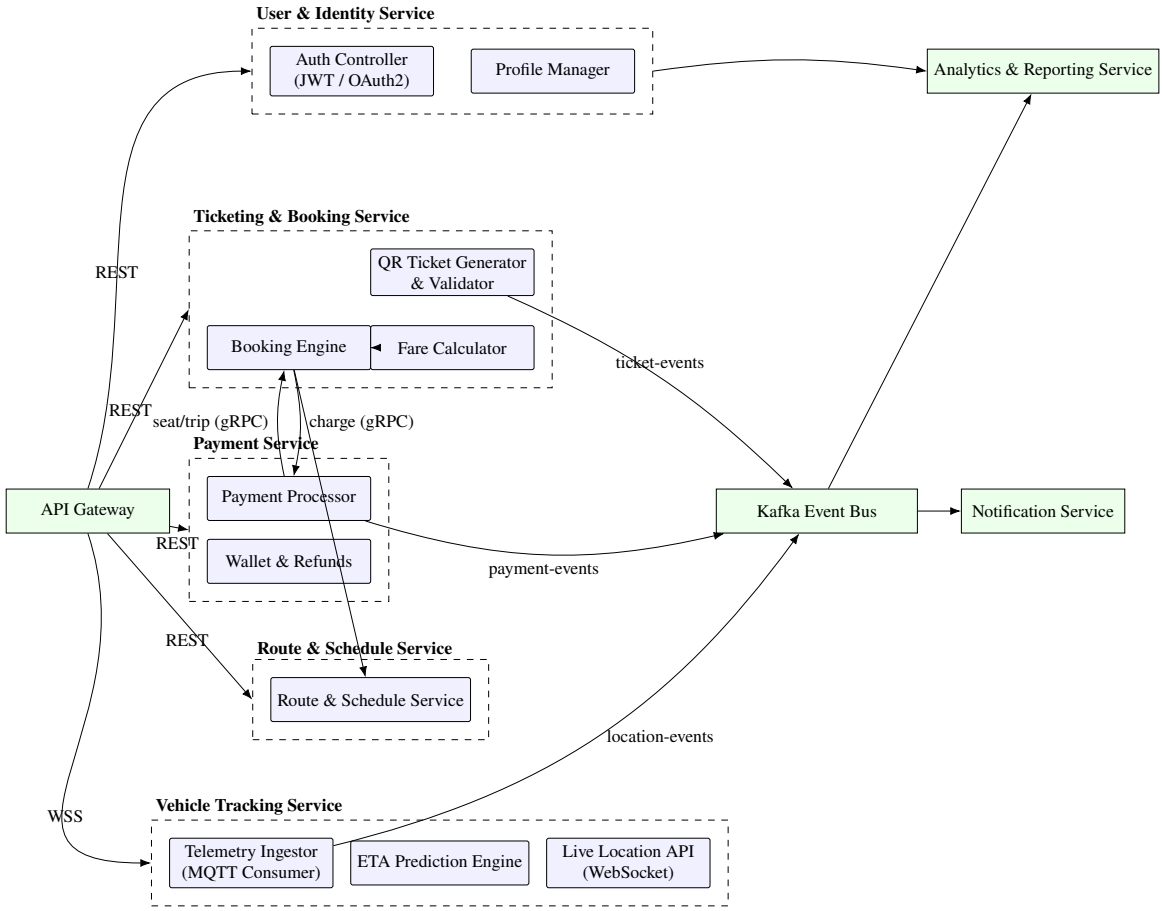


Figure 3: Component diagram showing internal components and interfaces

4.4 Deployment Diagram

TransitLK is deployed on AWS in the ap-south-1 (Mumbai) region, the closest region to Sri Lanka, across at least two availability zones. Client traffic enters through CloudFront with an attached web application firewall, reaches an application load balancer, and is served by service pods on an EKS Kubernetes cluster with horizontal pod autoscaling. GPS units publish MQTT over TLS to AWS IoT Core, which streams telemetry into the tracking service. Managed data services (Multi-AZ RDS PostgreSQL, ElastiCache Redis, MSK Kafka, S3) reduce operational burden and improve availability.

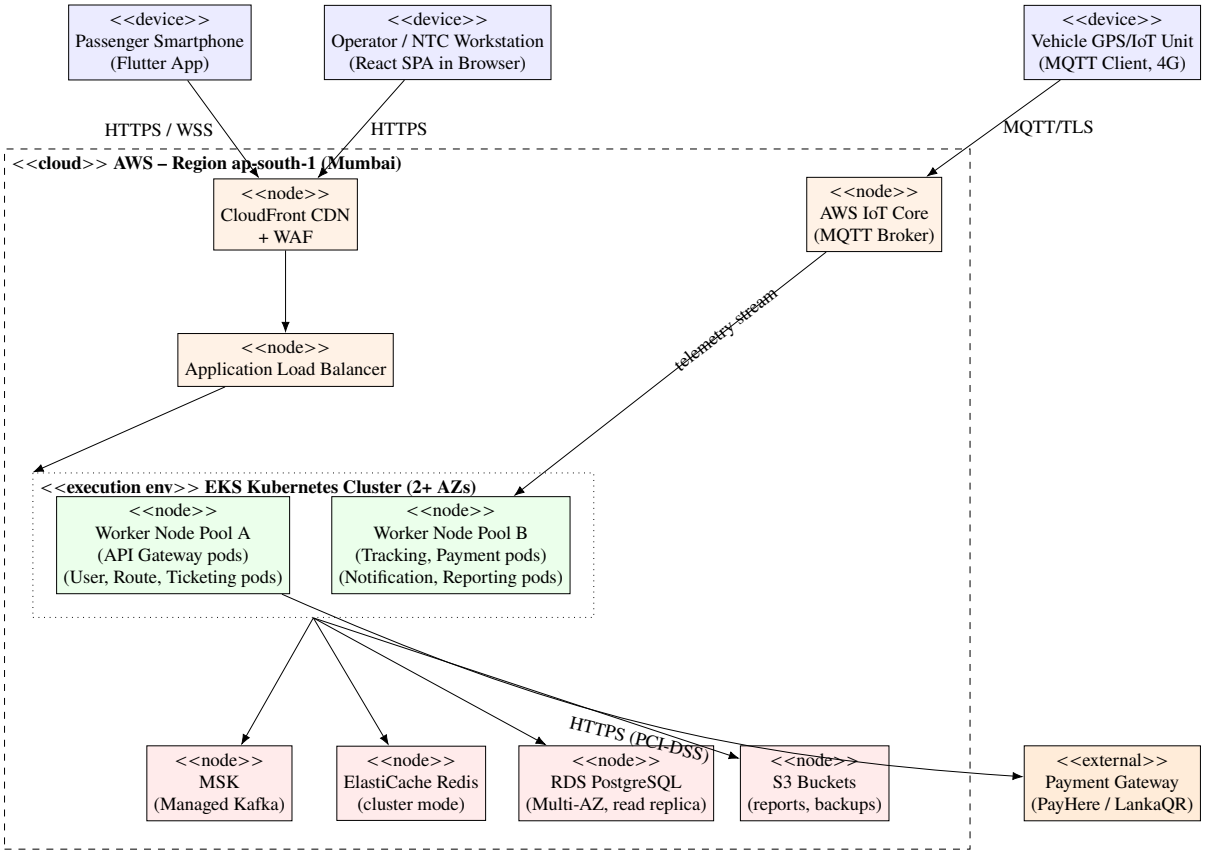


Figure 4: Deployment diagram on AWS with Kubernetes

4.5 Class Diagram

The class diagram models the core domain entities. Passenger and FleetOperator specialise User; operators own Vehicles, which are assigned to Trips scheduled on Routes; Tickets belong to Trips and are purchased by Passengers, each paid by exactly one Payment; Vehicles emit LocationPings that drive the live tracking and ETA features.

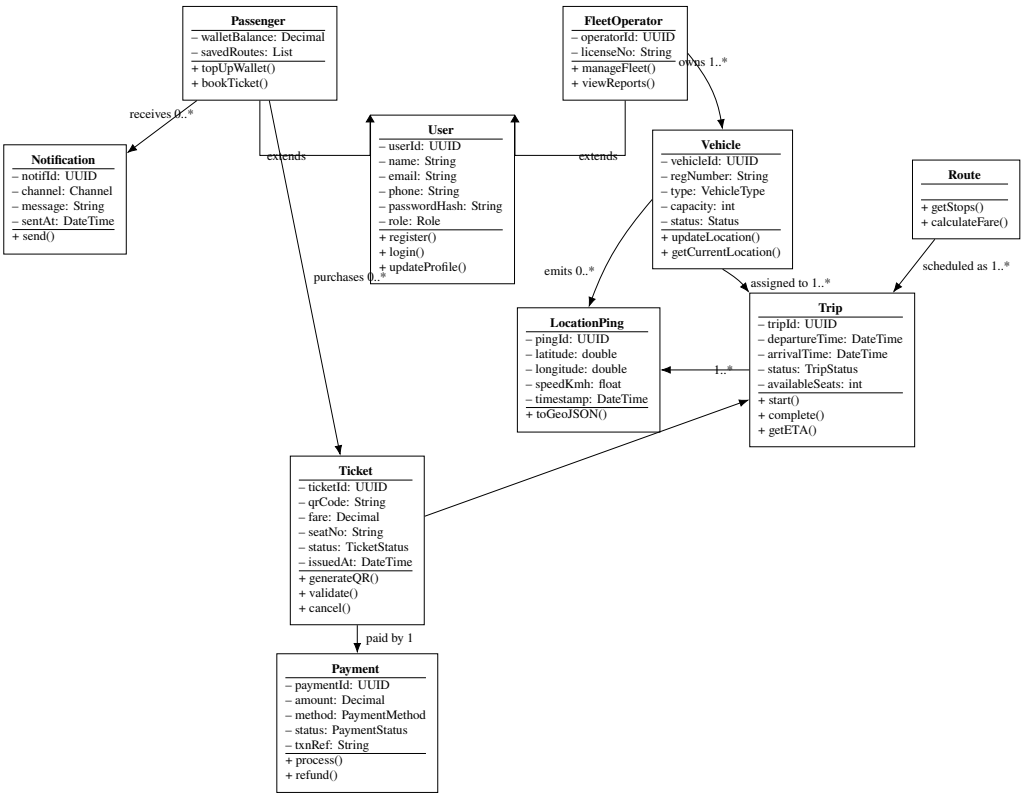


Figure 5: Class diagram of the core domain model

4.6 Data Flow Diagram (Level 1)

The Level 1 DFD traces the principal data flows: authentication against the user store (1.0), route search over the trip store (2.0), booking (3.0) which invokes payment processing (4.0) before persisting and issuing the QR e-ticket, continuous telemetry ingestion and ETA computation (5.0), and onboard ticket validation (6.0) which marks tickets as used to prevent reuse.

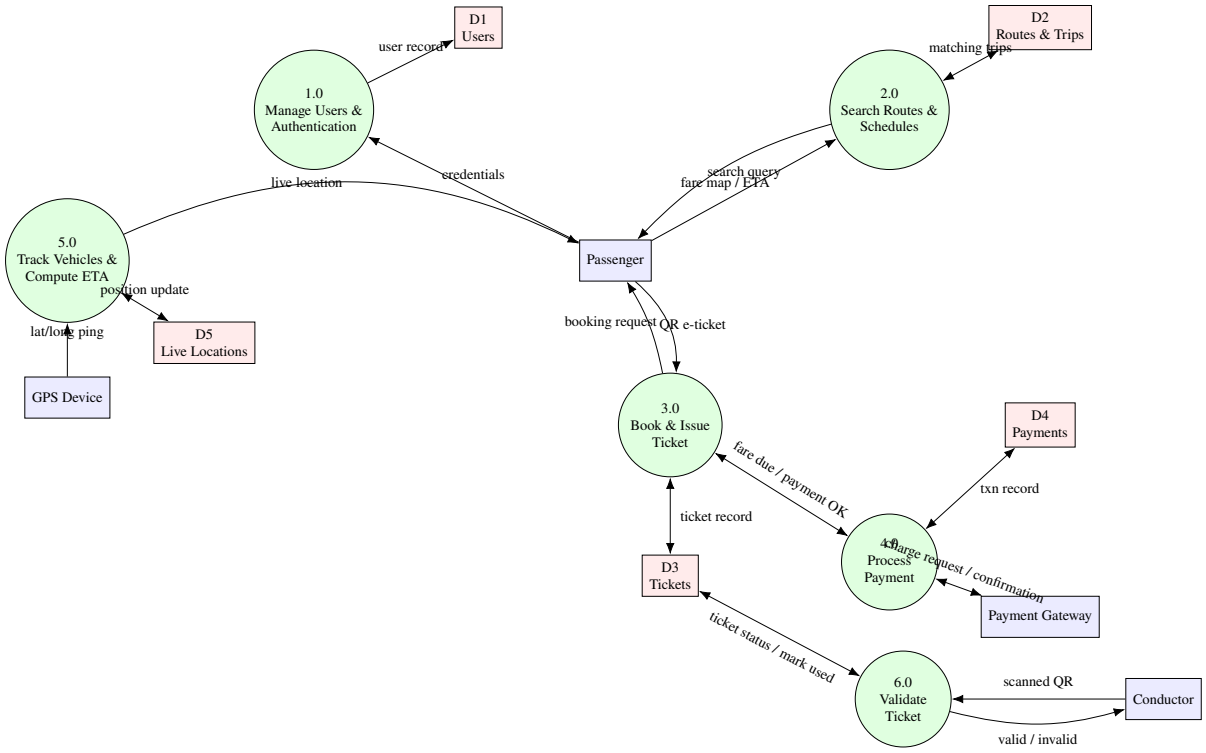


Figure 6: Level 1 data flow diagram

5. Technology Stack Selection

Technologies were selected to match the architecture style, the skills commonly available to development teams, and the managed services offered by the chosen cloud provider.

Layer	Technology	Rationale
Mobile apps (passenger, conductor)	Flutter (Dart)	Single codebase for Android and iOS; strong offline support needed by the validator app.
Web portal & dashboards	React + TypeScript	Mature ecosystem, component reuse, strong typing for maintainability.
API gateway	Kong on Kubernetes	Central authentication, rate limiting, routing, and observability.
Core services (ticketing, users, routes, payments)	Spring Boot (Java 21)	Robust transaction management, mature security libraries, wide developer availability.
Tracking & notification services	Node.js (NestJS)	Event-loop model efficiently handles thousands of concurrent WebSocket and stream connections.
Inter-service sync calls	gRPC / REST (OpenAPI)	Low-latency internal calls; standard, documentable public APIs.
Event backbone	Apache Kafka (AWS MSK)	Durable, replayable, high-throughput event streaming for telemetry and business events.
Telemetry ingestion	MQTT via AWS IoT Core	Lightweight publish/subscribe protocol suited to constrained 4G vehicle devices.
Transactional database	PostgreSQL (RDS Multi-AZ)	ACID guarantees for tickets and payments; PostGIS extension for geospatial queries.
Live state & cache	Redis (ElastiCache)	Sub-millisecond reads of live vehicle positions, sessions, and seat holds.
Telemetry history	TimescaleDB	Efficient time-series storage and aggregation of location pings.
Object storage	Amazon S3	Durable storage for reports, receipts, and backups.

Layer	Technology	Rationale
Orchestration & delivery	Docker, Kubernetes (EKS), GitHub Actions CI/CD	Independent deployment, autoscaling, self-healing, and automated pipelines.
Observability	Prometheus, Grafana, ELK, OpenTelemetry	Metrics, dashboards, centralised logs, and distributed tracing across services.
Payments	PayHere / bank gateways, LankaQR	Locally certified, PCI-DSS-compliant processing without storing card data.

Table 4: Technology stack selected for TransitLK

6. Quality Attributes

6.1 Scalability

Each microservice is packaged as a container and scaled independently by the Kubernetes horizontal pod autoscaler, so a morning-peak surge in bookings scales only the ticketing and payment services, while a growing vehicle fleet scales only the tracking pipeline. Kafka partitions distribute telemetry across consumer instances, Redis cluster mode shards live-location state, and read replicas offload reporting queries from the transactional database. Because services are stateless (state lives in Redis/PostgreSQL), new instances can join or leave at any time, giving near-linear horizontal scalability.

6.2 Performance

Latency-critical paths are engineered explicitly. Live vehicle positions are served from in-memory Redis rather than the relational database, and pushed to clients over WebSockets to avoid polling. Fare and schedule lookups are cached at the gateway and service level. Internal service-to-service calls use gRPC with protocol buffers to minimise serialisation overhead, and the CDN serves static assets close to users. Asynchronous processing via Kafka keeps slow work (report generation, notification fan-out) off the critical request path, supporting the 500 ms API and 3 s map-update targets of NFR1.

6.3 Security

Security is applied in depth. All external traffic uses TLS 1.3 terminated at the edge, with a WAF filtering common attacks. The API gateway enforces OAuth 2.0/JWT authentication

and role-based access control before requests reach any service. Passwords are hashed with bcrypt; payment card data never touches TransitLK servers because gateways return tokens (PCI-DSS SAQ-A scope). QR e-tickets are digitally signed with the ticketing service's private key so validators can verify authenticity even offline, and single-use semantics prevent replay. Services run in isolated Kubernetes namespaces with network policies, secrets are stored in a managed vault, and audit logs capture all administrative actions.

6.4 Availability

The deployment spans multiple availability zones: the load balancer, Kubernetes worker nodes, Kafka brokers, and the Multi-AZ PostgreSQL primary/standby pair all survive the loss of a data centre. Kubernetes restarts failed pods automatically and rolling updates avoid downtime during releases. Critically, ticket validation degrades gracefully — because QR tickets are cryptographically signed, the conductor app can verify them offline during connectivity loss and reconcile later — so revenue-critical operations continue even during partial outages, supporting the 99.9% availability target.

6.5 Maintainability

The database-per-service, bounded-context decomposition means each service can be understood, tested, and modified in isolation; adding a new payment method changes only the payment service. Contracts are made explicit through OpenAPI and protobuf definitions, and consumer-driven contract tests catch breaking changes in CI. Automated pipelines build, test, and deploy each service independently, and uniform observability (metrics, logs, traces) shortens diagnosis time. This directly supports NFR6 and long-term evolution of the platform.

6.6 Reliability

Reliability is achieved through redundancy and disciplined failure handling. Kafka provides durable, replayable event storage so telemetry or notification consumers can recover after crashes without data loss; consumers are idempotent to tolerate at-least-once delivery. The booking–payment interaction uses the saga pattern with compensating transactions, ensuring a passenger is never charged without receiving a ticket (and vice versa). Circuit breakers and timeouts prevent a slow payment gateway from cascading failures into the booking flow, retries use exponential backoff, and automated backups with point-in-time recovery meet the RPO/RTO targets of NFR5.

7. Conclusion

This report presented TransitLK, a cloud-native platform that addresses the most persistent problems of Sri Lankan public transport: unpredictable waiting times, cash-bound ticketing,

and the absence of operational data. Requirements analysis identified ten core functional requirements and eight measurable non-functional requirements across a diverse stakeholder set spanning passengers, conductors, fleet operators, the regulator, and payment providers.

After evaluating monolithic layered, client–server, event-driven, and microservices styles, a microservices architecture with an event-driven backbone was selected as the only style that simultaneously satisfies the demands of independent scalability, fault isolation, and continuous telemetry ingestion. The design decomposes the system into seven bounded-context services behind an API gateway, uses Kafka to decouple high-volume event flows, and applies polyglot persistence matched to each data access pattern. Deployment on AWS with Kubernetes across multiple availability zones, together with signed offline-verifiable QR tickets, saga-based payment consistency, and defence-in-depth security, ensures the quality attributes of scalability, performance, security, availability, maintainability, and reliability are addressed by concrete, verifiable mechanisms rather than assumptions.

The proposed architecture provides a solid foundation for the subsequent phases of this assessment: the individual written assessment in CA-02 and the functional prototype in CA-03, in which the key services — user management, ticketing with QR validation, and live tracking — will be implemented to demonstrate the architectural concepts in practice.